

作业3 Unix V6++, malloc、free 和 栈自动扩展

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博

学号 2351520

一、malloc 和 free

1、现运行进程 PA，1 页代码，1 页数据，没有只读数据 和 bss，1 页堆栈。代码段起始 0x401000。进程依次执行下列动态内存分配释放操作。

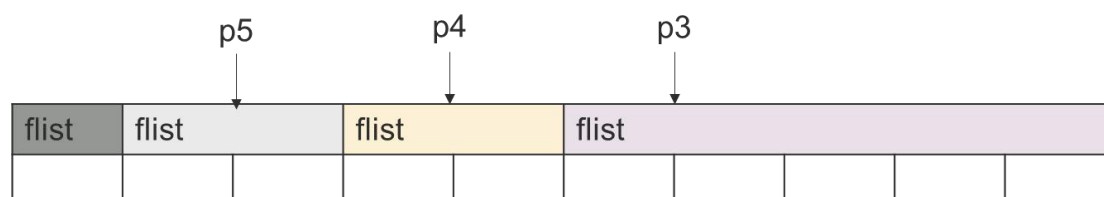
```
char *p1= malloc(4);      (1) 指针 p1 的值是多少？
char *p2= malloc(4);      (2) 指针 p2 的值是多少？
char *p3= malloc(32);     (3) 指针 p3 的值是多少？
free(p2);
char *p4= malloc(8);      (4) 指针 p4 的值是多少？
free(p1);
char *p5= malloc(8);      (5) 指针 p5 的值是多少？ (6) 情景分析
                        (7) 画最终的堆结构图
```

参考答案：

p1 = 0x403010 (内存片浪费 4 字节)
p2 = 0x403020 (内存片浪费 4 字节)
p3 = 0x403030
p4 = 0x403020 (p2 释放的内存片刚好够用)
p5 = 0x403010 (同上)

p5 执行 malloc，发现哑元和分配给 p4 的内存片之间的空闲区 (16 字节) 刚好够用，用该空闲区装新内存片 (没有浪费空间)，返回地址是 0x403010。

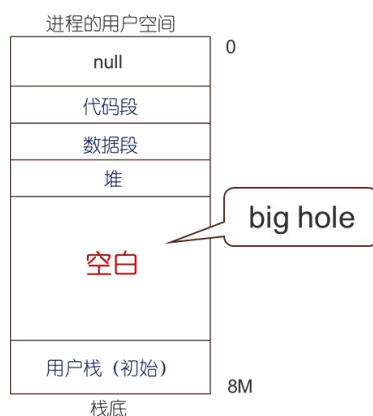
最终的堆结构图：



2、阅读 src/program/malloc.c。手推每块动态内存的起始地址，验证程序的输出。

- 3、阅读、逐行注释源代码 `src/lib/malloc.c`。阅读 `src/proc/Process.cpp`, `SBreak()` 函数。以应用程序 `src/program/malloc.c` 的执行过程为背景, 介绍 Unix V6++ 应用程序, 动态内存分配器的工作原理。
- 4、修改 `src/lib/malloc.c`, 将 `free()` 函数的时间复杂度降为 $O(1)$ 。

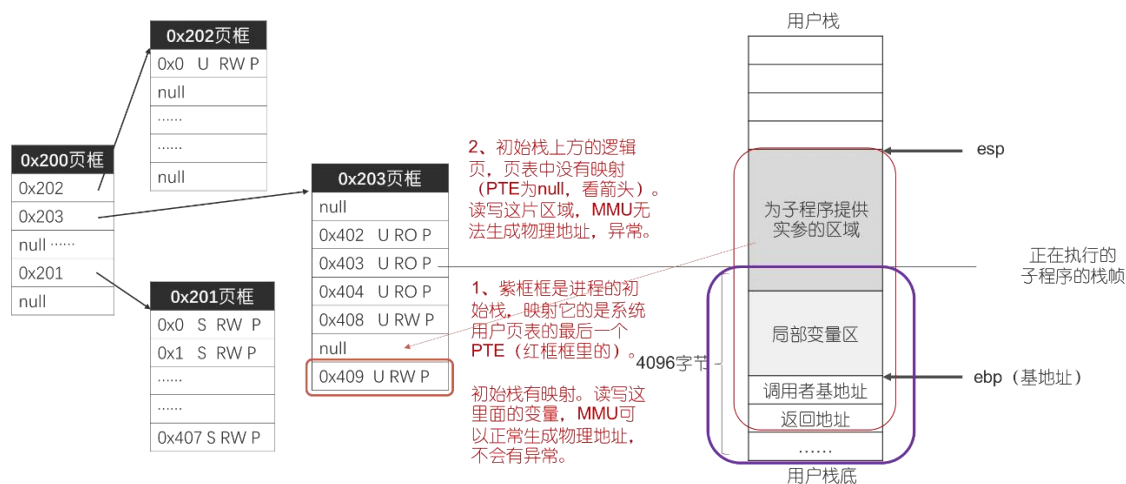
[跋] 每个需要使用动态内存的进程, 有一个运行在用户空间的动态内存分配器。`malloc()` 和 `free()`, 维护 `flist` 列表, 管理从 `big hole` 批发来的用户堆空间。



推荐, 阅读《深入理解计算机系统》 section 9.9 动态内存分配。这部分独立成篇, 可以不看前后章节。

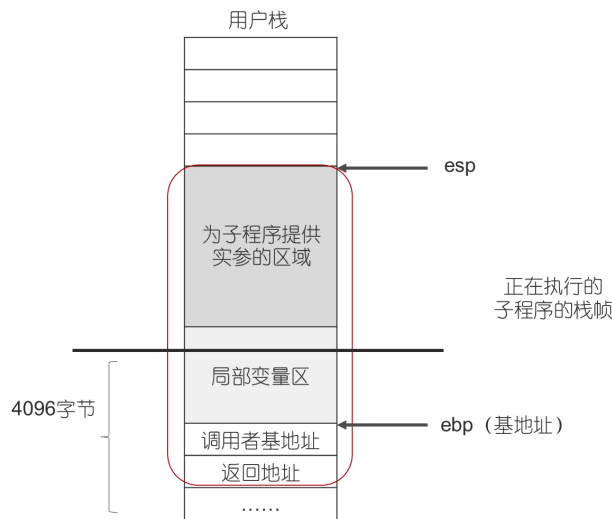
二、栈扩展

1、栈扩展, 判断条件



如图, 试问, 现在正在执行的子程序

- (1) 访问局部变量, MMU 会抛出缺页异常嘛?
不会。因为这里有映射。
- (2) 为子程序提供实参, ~~~~?
会。因为, 实参写在 `esp` 下面一点点。这里没有映射。
- (3) 执行 `call` 指令, ~~~~?
- (4) 调用的子程序执行指令 `push %ebp`, MMU 会抛出缺页异常吗? 子程序 访问局部变量, MMU 会抛出缺页异常吗?



如图，试问，图示栈帧

- (1) 构造阶段，MMU 会抛出缺页异常嘛？
- (2) 对应的子程序访问局部变量，MMU 会抛出缺页异常嘛？

2、完成以下任务

- (1) 阅读 src/program/stack.c, 观察堆栈扩展过程。



- (2) 堆栈扩展，判断条件是否合理？给出理由。

```
if( cr2 < MemoryDescriptor::USER_SPACE_SIZE - md.m_StackSize && cr2 >= context->esp - 8
    && md.m_DataSize + md.m_StackSize + PageManager::PAGE_SIZE <
        MemoryDescriptor::USER_SPACE_SIZE - md.m_DataStartAddress )
    current->SSStack();
```

// 3.1 堆栈扩展

- (3) $cr2 \geq context \rightarrow esp - 4$ ，可不可以？
- (4) $cr2 \geq context \rightarrow esp$ ，可不可以？

[跋] 栈是编译器和内核管的，编程的我们不需要操心。内存是宝贵的资源，先给每个进程少分点儿，真的不够用了，系统就悄悄地，后台帮你扩展。整个过程，应用程序无感，所以栈扩展和堆不一样，是自动的。